

Atividade 02 - Implementação Intercalação Balanceada

Implemente, usando arquivos para simular as fitas de entrada e saída, o método de ordenação externa Intercalação Balanceada de Vários Caminhos. Considere como entrada um arquivo conforme ilustrado a seguir (podendo variar os valores e quantidades) com um primeiro número indicando o tamanho da memória interna, um segundo número indicando a quantidade de fitas disponíveis e o conjunto de dados na sequência. Monte um arquivo PDF contando resumidamente como fez a atividade, quais problemas teve e como os superou, colocando prints do código e do resultado para ilustrar o que foi construído.

Exemplo de entrada:

3

6

ALGORITMOSEESTRUTURASDEDADOSIII

Relato: Durante o desenvolvimento do algoritmo, a primeira parte foi desenvolvida com certa tranquilidade. Mas a partir da intercalação propriamente dita, encontrei diversas dificuldades e necessitei de usar IA para desenvolver o restante. A maior dificuldade foi em como fazer a troca entre as fitas de leitura e saída, assim como, de que maneira realizar a ordenação alternando com os dados entre uma fita e outra. Também, devido ao uso de funções e bibliotecas pouco praticadas, teve-se um grande empecilho no desenvolvimento.

CÓDIGO:

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <fstream>
#include <cstdio>

using namespace std;

bool is_file_empty(const string &filename)
{
    ifstream file(filename, ios::ate); // Abre o arquivo e
    posiciona no final
    if (!file.is_open())
```

```

        return true; // Se não puder abrir, considere vazio/erro
        return file.tellg() == 0;
    }

void mostrarConteudoDasFitas(int qtd_fitas, const vector<string>
&nomes_fitas)
{
    for (int i = 0; i < qtd_fitas; ++i)
    {
        cout << "Fita " << i + 1 << " (" << nomes_fitas[i] << "):
";
        ifstream fita_para_leitura(nomes_fitas[i]);

        if (fita_para_leitura.is_open())
        {
            string bloco;
            while (getline(fita_para_leitura, bloco))
            {
                cout << bloco << " | ";
            }
            fita_para_leitura.close();
        }
        cout << endl;
    }
}

int main()
{
    int qtd_fitas, qtd_memoria;
    string dado = "";
    ifstream arq_instrucoes("instrucoes.txt", ios::in);
    if (!arq_instrucoes.is_open())
    {
        cout << "Erro ao abrir o arquivo 'instrucoes.txt'!" <<
endl;
        return 1;
    }
    arq_instrucoes >> qtd_memoria;
    arq_instrucoes.ignore();
    arq_instrucoes >> qtd_fitas;
    arq_instrucoes.ignore();
    getline(arq_instrucoes, dado);
    arq_instrucoes.close();

    cout << "Tamanho da memoria interna: " << qtd_memoria <<
endl;
    cout << "Quantidade de fitas: " << qtd_fitas << endl;
    cout << "O tamanho da palavra e " << dado.length() << "." <<
endl;

    if (qtd_fitas % 2 != 0)
    {
        cout << "Erro: A quantidade de fitas deve ser um numero
par." << endl;
        return 1;
    }
}

```

```

    }

    // --- Preparação dos Nomes de Arquivo ---
    vector<string> nomes_fitras;
    for (int i = 0; i < qtd_fitras; ++i)
    {
        nomes_fitras.push_back("fita" + to_string(i + 1) +
".txt");
        // Limpa arquivos de execuções anteriores
        remove(nomes_fitras[i].c_str());
    }

    // --- Fase de Distribuição Inicial (Escrevendo nos arquivos)
    ---
    int k = qtd_fitras / 2;
    { // Bloco de escopo para garantir que os arquivos sejam
    fechados
        vector<ofstream> fitas_saida(k);
        for (int i = 0; i < k; ++i)
        {
            fitas_saida[i].open(nomes_fitras[i]);
        }

        int fita_atual = 0;
        for (size_t i = 0; i < dado.length(); i += qtd_memoria)
        {
            vector<char> memoria_interna;
            for (int j = 0; j < qtd_memoria && (i + j) <
dado.length(); ++j)
            {
                memoria_interna.push_back(dado[i + j]);
            }
            sort(memoria_interna.begin(), memoria_interna.end());

            string bloco(memoria_interna.begin(),
memoria_interna.end());
            fitas_saida[fita_atual] << bloco << endl; // Escreve
o bloco no arquivo, um por linha

            fita_atual = (fita_atual + 1) % k;
        }
    } // Os destrutores de ofstream fecham os arquivos aqui

    cout << "\n--- Conteudo das fitas apos a distribuicao inicial
---\n";
    mostrarConteudoDasFitas(qtd_fitras, nomes_fitras);

    // --- Fase de Intercalação Balanceada (Lendo e escrevendo em
arquivos) ---
    bool usar_primeira_metade = true;
    int rodada = 0;

    while (true)
    {
        // Checar a condição de parada ANTES de iniciar a rodada

```

```

int fitas_com_dados = 0;
for (int i = 0; i < qtd_fitas; ++i)
{
    if (!is_file_empty(nomes_fitas[i]))
    {
        fitas_com_dados++;
    }
}
if (fitas_com_dados <= 1)
{
    cout << "\n--- Fim da intercalacao ---";
    break;
}

cout << "\n--- Iniciando rodada " << rodada + 1 << "
---\n";

int inicio_leitura, inicio_escrita;
if (usar_primeira_metade)
{
    inicio_leitura = 0;
    inicio_escrita = k;
    cout << "Lendo das fitas " << inicio_leitura + 1 <<
    "-" << inicio_leitura + k << " e escrevendo nas fitas " <<
    inicio_escrita + 1 << "-" << inicio_escrita + k << endl;
}
else
{
    inicio_leitura = k;
    inicio_escrita = 0;
    cout << "Lendo das fitas " << inicio_leitura + 1 <<
    "-" << inicio_leitura + k << " e escrevendo nas fitas " <<
    inicio_escrita + 1 << "-" << inicio_escrita + k << endl;
}

// Abrir os arquivos de leitura e escrita para esta
rodada
vector<ifstream> fitas_leitura(k);
vector<ofstream> fitas_escrita(k);
for (int i = 0; i < k; ++i)
{
    fitas_leitura[i].open(nomes_fitas[inicio_leitura +
i]);
    fitas_escrita[i].open(nomes_fitas[inicio_escrita +
i]); // Abre em modo de escrita (truncando)
}

int fita_saida_atual = 0;
while (true)
{
    vector<string> blocos_para_intercalar;
    bool ha_blocos_para_ler = false;

    // Tenta ler um bloco de cada fita de entrada
    for (int i = 0; i < k; ++i)

```

```

        {
            string bloco;
            if (getline(fitas_leitura[i], bloco))
            {
                blocos_para_intercalar.push_back(bloco);
                ha_blocos_para_ler = true;
            }
            else
            {
                // Adiciona um placeholder se a fita acabou,
para manter os índices alinhados
                blocos_para_intercalar.push_back("");
            }
        }

        if (!ha_blocos_para_ler)
        {
            break; // Nenhuma fita de entrada tem mais blocos
        }

        // Lógica de intercalação (k-way merge), idêntica à
original
        string bloco_intercalado = "";
        vector<int>
ponteiros_de_char(blocos_para_intercalar.size(), 0);

        while (true)
        {
            char min_char = '~'; // Caractere com valor ASCII
alto
            int indice_bloco_com_min = -1;

            for (int i = 0; i <
blocos_para_intercalar.size(); ++i)
            {
                if (ponteiros_de_char[i] <
blocos_para_intercalar[i].length())
                {
                    if
(blocos_para_intercalar[i][ponteiros_de_char[i]] < min_char)
                    {
                        min_char =
blocos_para_intercalar[i][ponteiros_de_char[i]];
                        indice_bloco_com_min = i;
                    }
                }
            }

            if (indice_bloco_com_min == -1)
            {
                break; // Todos os blocos foram consumidos
            }

            bloco_intercalado += min_char;
            ponteiros_de_char[indice_bloco_com_min]++;
        }
    }
}

```

```

    }

    // Escreve o novo bloco intercalado em uma das fitas
de saída    fitas_escrita[fita_saida_atual] << bloco_intercalado
<< endl;    fita_saida_atual = (fita_saida_atual + 1) % k;
    }

    // Fecha e limpa os arquivos de leitura da rodada
for (int i = 0; i < k; ++i)
{
    fitas_leitura[i].close();
    fitas_escrita[i].close(); // Garante que tudo foi
escrito no disco
    // Limpa o conteúdo do arquivo de leitura
ofstream cleaner(nomes_fitas[inicio_leitura + i],
ios::trunc);
    cleaner.close();
}

    cout << "\n--- Conteúdo das fitas após a rodada " <<
rodada + 1 << " ---\n";
    mostrarConteudoDasFitas(qtd_fitas, nomes_fitas);

    rodada++;
    usar_primeira_metade = !usar_primeira_metade; // Alterna
para o próximo conjunto de fitas
}

// --- Impressão do Resultado Final ---
cout << "\n\nPalavra ordenada final:\n";
for (const auto &nome_fita : nomes_fitas)
{
    if (!is_file_empty(nome_fita))
    {
        cout << "Resultado encontrado em '" << nome_fita <<
":\n";

        ifstream fita_final(nome_fita);
        string bloco;
        while (getline(fita_final, bloco))
        {
            cout << bloco;
        }
        fita_final.close();
        break; // O resultado estará em apenas uma fita
    }
}
cout << endl
    << endl;

return 0;
}

```

RESULTADO:

```
Tamanho da memoria interna: 3
Quantidade de fitas: 6
O tamanho da palavra e 31.

--- Conteudo das fitas apos a distribuicao inicial ---
Fita 1 (fita1.txt): AGL | EES | ARS | IIS |
Fita 2 (fita2.txt): IOR | RST | DDE | I |
Fita 3 (fita3.txt): MOT | TUU | ADO |
Fita 4 (fita4.txt):
Fita 5 (fita5.txt):
Fita 6 (fita6.txt):

--- Iniciando rodada 1 ---
Lendo das fitas 1-3 e escrevendo nas fitas 4-6

--- Conteudo das fitas apos a rodada 1 ---
Fita 1 (fita1.txt):
Fita 2 (fita2.txt):
Fita 3 (fita3.txt):
Fita 4 (fita4.txt): AGILMOORT | IIIS |
Fita 5 (fita5.txt): EERSSTTUU |
Fita 6 (fita6.txt): AADDDEORS |

--- Iniciando rodada 2 ---
Lendo das fitas 4-6 e escrevendo nas fitas 1-3

--- Conteudo das fitas apos a rodada 2 ---
Fita 1 (fita1.txt): AAADDDEEGILMOOORRRSSSTTTUU |
Fita 2 (fita2.txt): IIIS |
Fita 3 (fita3.txt):
Fita 4 (fita4.txt):
Fita 5 (fita5.txt):
Fita 6 (fita6.txt):

--- Iniciando rodada 3 ---
Lendo das fitas 1-3 e escrevendo nas fitas 4-6

--- Conteudo das fitas apos a rodada 3 ---
Fita 1 (fita1.txt):
Fita 2 (fita2.txt):
Fita 3 (fita3.txt):
Fita 4 (fita4.txt): AAADDDEEGIIIIILMOOORRRSSSTTTUU |
Fita 5 (fita5.txt):
Fita 6 (fita6.txt):

--- Fim da intercalacao ---

Palavra ordenada final:
Resultado encontrado em 'fita4.txt':
AAADDDEEGIIIIILMOOORRRSSSTTTUU
```